

과제 1

탁구 게임기를 마이크로컨트롤러에서 C++로 구현하면, CPU가 버튼 입력 확인, 공의 위치 계산, 충돌 판정, 화면 출력 등의 명령을 순서대로 실행한다. 이 작업들이 빠르게 반복되기 때문에 사용자에게는 동시에 이루어지는 것처럼 보이지만, 하나의 CPU 코어에서는 기본적으로 명령을 차례대로 처리한다.

반면 Verilog 같은 하드웨어 기술 언어는 동시에 동작하는 회로들의 구조와 동작을 기술한다. 예를 들어 버튼 입력을 감지하는 회로, 공의 위치를 계산하는 회로, 화면 출력 신호를 만드는 회로를 각각 구성하면 이 회로들은 병렬로 동작할 수 있다. 따라서 화면 출력 회로가 동작하는 동안에도 다른 회로가 버튼 입력을 처리할 수 있다.

과제 2

래스터 주사는 화면을 왼쪽에서 오른쪽으로, 위에서 아래로 줄 단위로 훑으며 영상을 표시하는 방식이다. CRT에서는 전자빔이 화면을 훑으며 빛을 내고, 디스플레이 패널에서는 화소를 전기적으로 제어하여 영상을 표시한다.

래스터 주사에서 줄을 표시하는 순서에 따라 순차 주사와 비월 주사로 나뉜다.

순차주사는 1,2,3,4,...번째 줄을 차례대로 표시하고 비월 주사는 홀수 줄을 먼저 끝까지 표시한 뒤 짝수 줄을 표시한다. 각 줄 묶음을 필드라고 하며, 두 필드가 한 프레임을 구성한다.

HSYNC는 각 줄의 수평 타이밍을 맞추고, VSYNC는 화면 위쪽부터 다시 표시하는 수직 타이밍을 맞춘다. 순차 주사에서는 프레임마다, 비월 주사에서는 필드마다 수직 동기를 맞춘다. 실제 화소의 색과 밝기는 영상 데이터가 결정한다.

예시)순차 주사

1 → HSYNC → 2 → HSYNC → 3 → HSYNC → 4 → HSYNC → 5 → HSYNC → 6 → HSYNC → 7 → HSYNC → 8 → VSYNC → 다시 1...

예시)비월 주사

1 → HSYNC → 3 → HSYNC → 5 → HSYNC → 7 → VSYNC → 2 → HSYNC → 4 → HSYNC → 6 → HSYNC → 8 → VSYNC → 다시 1...

과제 3

과제 4

탁구 게임기는 마이크로컨트롤러의 소프트웨어로도 충분히 구현할 수 있다. 하지만 전용 칩으로 설계하면 게임에 필요한 기능을 회로로 직접 구성하여 여러 작업을 동시에 처리할 수 있다는 장점이 있다. 예를 들어 버튼 입력 감지, 공의 위치 계산, 충돌 판정, 화면 출력 등을 각각의 회로에서 병렬로 수행할 수 있다.

또한 범용 프로세서처럼 명령을 반복해서 읽고 해석하는 과정 없이 필요한 기능을 직접 수행하므로, 설계에 따라 소비 전력과 처리 지연을 줄일 수 있다. 대량 생산할 경우에는 필요한 기능을 하나의 칩에 통합하여 제품의 크기와 개당 비용을 줄일 가능성도 있다. 다만 전용 칩은 초기 개발 비용이 크고, 제작 후 기능을 수정하기 어렵다는 단점이 있다.

과제 5

C++에서 ++yPos와 yPos++는 모두 yPos의 값을 1 증가시키지만, 표현식에서 사용하는 값이 다르다. ++yPos는 증가한 값을 사용하고, yPos++는 증가하기 전의 값을 사용한다.

예)

```
int yPos = 5;
```

```
int a = ++yPos; // yPos = 6, a = 6
```

예)

```
int yPos = 5;
```

```
int a = yPos++; // yPos = 6, a = 5
```

C++의 순차 실행에서는 앞 문장의 처리가 끝난 뒤 다음 문장이 실행된다. 따라서 yPos++를 단독으로 실행하면 그다음 문장은 증가한 값을 사용한다. 단독 문장으로 사용한 ++yPos;와 yPos++;의 결과는 같다. 후위 증가라고 해서 다음 문장까지 증가가 미뤄지는 것은 아니다.

베릴로그에서는 여러 always 블록이나 assign 구문이 서로 병렬로 동작하는 회로를 나타낸다. 하나의 절차적 블록 내부에는 구문의 실행 순서가 있으며, 모든 구문이 무조건 동시에 실행되는 것은 아니다.

이러한 환경에서 ++, --처럼 값을 참조하면서 변경하는 연산자를 사용하면, 다른 블록이 변경 전의 값을 읽는지 변경 후의 값을 읽는지가 문제가 될 수 있다. 예를 들어 같은 클록에서 한 블록이 yPos를 즉시 증가시키고 다른 블록이 이를 읽으면, 블록의 실행 순서에 따라 읽는 값이 달라지는 경쟁 상태가 발생할 수 있다. 경쟁상태의 문제를 방지하기 위해 클록에 따라 동작하는 레지스터는 다음처럼 논블로킹 대입(<=)으로 표현한다

예)

```
ypos =5라면
```

```
논블로킹 대입
```

```
yPos <= yPos + 1;
```

```
a <= yPos; // a에는 5가 저장됨
```